

Java Notes

Lecture-2: The History and Overview of Java

Q1: Definitions:

Encapsulation: Encapsulation is the mechanism that binds together code and the data it manipulates and keeps both safe from outside interference and misuse.

Inheritance: Inheritance is the process by which one object acquires the properties of another object. This is important because it supports the concept of hierarchical classification.

Polymorphism: Polymorphism (from Greek, meaning “many forms”) is a feature that allows one interface to be used for a general class of actions.

Q2: Difference between c++ and java.

	C++	Java
Memory management	Use a pointers, structures and union	No use of pointers supports reference trade and interfaces
Platform	Dependent platform	Independent Platform
Libraries	available with low level functionalities	Wide range of classes for various high-level services
Inheritance	Provide both single and multiple inheritance	Provide only Multiple inheritance.
Operator overloading	Support operator overloading	It does not support this feature
Program handling	Functions and variable can reside outside classes	Functions and variables we site only in classes packages are used
Portability	Platform depends must be recombined for different platform	Independent by code generated wives on every os
Garbage collection	Programmer responsible for garbage collection	It has auto garbage collection

Topic	C++	Java
Memory Management	Use of pointers, structures, union	No use of pointers. Supports references, thread and interfaces
Libraries	Comparatively available with low-level functionalities	Wide range of classes for various high-level services

Multiple Inheritance	Provide both single and multiple inheritance.	Multiple inheritances is partially done through interfaces
Operator Overloading	Supports operator overloading	It doesn't support this feature
Program Handling	Platform dependent, must be recompiled for different Platform	Platform independent, byte code generated works on every OS.
Portability	Functions and variables can reside outside classes.	Functions and variables reside only in classes, packages are used
Thread Support	No built-in support for threads, depends on libraries.	It has built-in thread support.
Garbage Collection	Object Programmer responsible for Garbage Collection	It has Garbage Collection

Q3. Run time error and compile time error.

a) Write down the output of the below code:

```
public class Example {
    public static void main(String[] args) {
        int number = 10;
        System.out.println("The number is: " + num);
    }
}
```

Answer: Error: Compile-Time Error. Variable `num` is not defined

b) Write down the output of the below code:

```
public class Example {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3};
        System.out.println(arr[5]);
    }
}
```

Answer: Error: Runtime Error (ArrayIndexOutOfBoundsException)

c) Write down the output of the below code:

```
public class Example {
    public static void main(String[] args) {
```

```

        System.out.println("Hello World")
    }
}

```

Answer: Error: Compile-Time Error, Missing semicolon.

d) Write down the output of the below code:

```

public class Example {
    public static void main(String[] args) {
        int result = 10 / 0;
        System.out.println(result);
    }
}

```

Answer: Error: Runtime Error, Arithmetic Exception(Divide by zero).

e) Write down the output of the below code:

```

public class Example {
    public static void main(String[] args) {
        String str = null;
        System.out.println(str.length());
    }
}

```

Answer: Error: Logical Error Leading to Runtime Error, (NullPointerException)

Lecture-3: Java Basic and Syntax

Q1: Definitions:

Java Virtual Machine (JVM): The Java Virtual Machine (JVM) is an abstract machine that provides a runtime environment to execute Java bytecode. It acts as a bridge between the compiled Java program and the underlying host operating system. The JVM manages memory (stack, heap, etc.), performs

garbage collection, and ensures platform independence by abstracting the hardware.

JDK: It is a software development kit requires to java applications. It includes -
Java Runtime Environment (JRE) + Development tools

JRE: Java Virtual Machine (JVM) + Libraries to run the application

White Space: In Java, **whitespace** refers to any character or set of characters that are used for spacing but are not visible when rendered.

Q2: Write down the difference between compiler and Interpreter.

Feature	Compiler	Interpreter
Translation process	Scan entire program fast then translates into machine code	scan the program line by line and translate into machine code
Debugging	Slower	Faster
Showing errors	All errors At a time	One error at the time
Error detection	Before execution	During execution
Execution time	Less	More
Flexibility	Less flexibility	More flexible
	C, C++	
Feature	Interpreter	Compiler
Translation Process	Scans and translates line by line	Scans the entire program, then translates
Error Reporting	Shows one error at a time	Shows all errors and warnings at once
Error Occurrence	Errors are found after each line	Errors are found after scanning the whole program
Debugging	Faster debugging	Slower debugging
Execution Time	Higher execution time	Lower execution time
Examples	Used by languages like Java, Python	Used by languages like C, C++

Q4. Variable name valid or not?

1. age
2. \$salary
3. _value
4. __1_value
5. 123abc

6. -salary
7. name1
8. firstName
9. boolean
- 10.\$\$_myVar
- 11.@age
- 12.number#
- 13._1234
- 14.class
- 15.TOTAL_VALUE

Answer:

1. age - Valid (starts with a letter).
2. \$salary - Valid (starts with a \$).
3. _value - Valid (starts with an underscore).
4. __1_value - Valid (starts with underscores, followed by letters/digits).
5. 123abc - Invalid (starts with a digit).
6. -salary - Invalid (contains a -, which is not allowed).
7. name1 - Valid (starts with a letter, contains letters and digits).
8. firstName - Valid (follows camel case naming convention).
9. boolean - Invalid (reserved keyword).
- 10.\$\$_myVar - Valid (starts with \$, contains valid characters).
- 11.@age - Invalid (contains @, which is not allowed).
- 12.number# - Invalid (contains #, which is not allowed).
- 13._1234 - Valid (starts with an underscore).
- 14.class - Invalid (reserved keyword).
- 15.TOTAL_VALUE - Valid (uses uppercase letters and an underscore).

Lecture-4: Variables and Data Types

Q1: Definitions:

String: The string is a class in Java, and it represents the sequences of characters.

Arrays: Arrays are created with the help of primitive data types and store multiple values of the same type.

Classes: The classes are the user-defined data types and consist of variables and methods.

Interfaces: The interfaces are abstract types that are used to specify a set of methods.

Q2: What are Local variables, Static variables and Class variable.

Answer: Local variables: Local variables are declared in methods, constructors, or blocks.

Example:

```
public void pupAge() {  
    int age = 0;  
    age = age + 7;  
    System.out.println("Puppy age is : " + age);  
}
```

Static variables: Instance variables are declared in a class, but outside a method, constructor or any block. Instance variables are created when an object is created with the use of the keyword 'new' and destroyed when the object is destroyed.

Example:

```
public class Employee {  
    public String name;  
    private double salary;  
}
```

Class variables: Class variables also known as static variables are declared with the static keyword in a class, but outside a method, constructor or a block. Static variables are stored in the static memory.

```
Example: public class Employee {  
    private static double salary;  
    public static final String DEPARTMENT = "Development ";  
}
```

Q3. Java type Casting.

Answer: Typecasting in Java is the process of converting one data type to another data type using the casting operator.

Syntax: variableName = () value;

There are two types of Type Casting in java:

1. **Widening Type Casting (Implicit):** A lower data type is transformed into a higher one by a process known as widening type casting.
Example: int i = 10;
 double d = i;
2. **Narrow Type Casting (Explicit):** The process of downsizing a bigger data type into a smaller one is known as narrowing type casting.
Example : double i = 100.245;
 short j = (short)i;
 int k = (int)i;

Q4. Write which type of typecasting is applied in the following code:

```
public class TypeCastingExample {  
    public static void main(String[] args) {  
        int i = 42;  
        double d = i;  
        System.out.println("Widening Type Casting: " + d);  
        double x = 99.99;  
        int y = (int) x;  
        System.out.println("Narrowing Type Casting: " + y);  
        // Identify the type of casting:  
        short s = 120;
```

```
long l = s; // Example A
float f = 123.45f;
byte b = (byte) f; // Example B
}
}
```

Answer:

Example A: Widening Type Casting (Implicit) – short to long.

Example B: Narrowing Type Casting (Explicit) – float to byte.

Lecture-5: Array in Java

Q1: What is an Array?

Ans: Array is used to store a collection of variables of the same type. Memory Space is allocated for values of a particular type by declaring an array.

Q2: Sort an array of string in reverse order.

Ans: Here is the code:

```
import java.util.Arrays;
import java.util.Collections;

public class ReverseSortArray {
    public static void main(String[] args) {
        // Array of strings
        String[] words = {"apple", "orange", "banana",
"grape", "mango"};

        // Sort in reverse order
        Arrays.sort(words, Collections.reverseOrder());

        // Print the sorted array
        System.out.println("Sorted array in reverse order:
" + Arrays.toString(words));
    }
}
```

Q3: Array Sorting in Ascending and Descending.

Ans: Here is the code:

```
import java.util.Arrays;
import java.util.Collections;

public class IntArraySortingExample {
    public static void main(String[] args) {
        // Array of integers
        Integer[] numbers = {5, 2, 8, 1, 3};

        // Sorting in ascending order
        Arrays.sort(numbers);
        System.out.println("Ascending order: " +
Arrays.toString(numbers));
    }
}
```

```

        // Sorting in descending order
        Arrays.sort(numbers, Collections.reverseOrder());
        System.out.println("Descending order: " +
Arrays.toString(numbers));
    }
}

```

Q4. Trace the following Java program based on the last two digits of your university ID.

public class ArrayCalculation {
public static void main(String[] args) {
int secondLast = 0; // Replace with the second-last digit of your ID
int last = 0; // Replace with the last digit of your ID
secondLast = (secondLast == 0 && last == 0) ? 1 : (secondLast == 0 ? 1 :
secondLast);
last = (secondLast == 1 && last == 0) ? 1 : last;
int size = secondLast + last; // Calculate array size
int[] arr = new int[size];
for (int i = 0; i < size; i++) {
arr[i] = last + i;
}
System.out.println("Array elements:");
for (int i = 0; i < size; i++) {
System.out.print(arr[i] + " ");
}
}

Answer: My Id is = 11230321539

secondLast = 3

last = 9

size = 12

Output:

Array elements: 9 10 11 12 13 14 15 16 17 18 19 20

Q5. Trace the following Java program

```

public class ArraySumCalculation {
    public static void main(String[] args) {
        int a = 5; // Replace with the first number of your birth year
        int b = 3; // Replace with the second number of your birth year

        int sum = (a + b) * 2; // Calculate sum
        int[] arr = new int[sum];

        for (int i = 0; i < sum; i++) {
            arr[i] = a * i + b;
        }

        int result = 0;
        for (int i = 0; i < sum; i++) {
            result += arr[i];
        }

        System.out.println("Sum of Array elements: " + result);
    }
}

```

Answer:

My Birth year = 2003

a = 2

b = 0

size = 2

Output:

Sum of Array elements: 6 (12 hobe).

Lecture-6, 7

Q1: What is operator?

Ans: An operator is a symbol that operates on one or more arguments to produce a result.

Q2: What is bitwise operator?

Ans: An operator is a symbol that operates on one or more arguments to produce a result.

Q3. What are control statements?

Answer: Control statements in Java are special statements that determine the flow of execution of a program. They allow developers to make decisions, loop through a block of code, or jump to another part of the program, based on conditions or specific requirements.

Q4. Increment decrement code tracing.

```
public class IncrementDecrement {  
    public static void main(String[] args) {  
        int a = 5, b = 10, c = 15;  
  
        a = b++ + ++c; // Step 1  
        b = --c - a--; // Step 2  
        c = a++ + b--; // Step 3  
  
        System.out.println("a: " + a);  
        System.out.println("b: " + b);  
        System.out.println("c: " + c);  
    }  
}
```

Write down the output of this code.

Answer:

a: 26
b: -12
c: 14

Lecture: 8

Q1. What is a Constructor?

Answer: A special method used to initialize objects when they are created.

Key Features:

- i. Same name as the class.
- ii. No return type (not even void).
- iii. Automatically called when the object is created.

Purpose of Constructors:

- i. Simplifies initialization by performing setup at creation time.
- ii. Ensures objects are fully initialized and usable immediately.

Benefits:

- i. Eliminates repetitive initialization code.
- ii. Replaces manual methods like setDim().

Q2. What is a Parameterized Constructor?

Answer: A constructor that accepts parameters to initialize an object with specific values.

Q3. Implement the missing pieces to make this code executable. Write down the required class, variables, and methods.

```
public static void main(String[] args) {  
    Calculator calc = new Calculator();  
    int sum = calc.add(5, 10);  
    double result = calc.multiply(4.5, 2);  
    calc.displayResult("Sum", sum);  
    calc.displayResult("Product", result);  
}
```

Answer:

```
class Calculator {  
    public int add(int a, int b) {  
        return a + b;  
    }  
    public double multiply(double a, double b) {  
        return a * b;  
    }  
}
```

```
    }  
    public void displayResult(String label, double value)  
{  
        System.out.println(label + ": " + value);  
    }  
}  
public class Main {  
    public static void main(String[] args) {  
        Calculator calc = new Calculator();  
        int sum = calc.add(5, 10);  
        double result = calc.multiply(4.5, 2);  
        calc.displayResult("Sum", sum);  
        calc.displayResult("Product", result);  
    }  
}
```

For Extra Practice:

Q1. Identify and correct the errors in each of the following statements:

a) `if (c < 7); System.out.println("c is less than 7");`

b) `if (c ==> 7) System.out.println("c is equal to or greater than 7");`

Answer:

a) Error: Semicolon after the right parenthesis of the condition (`c < 7`) in the `if`.
Correction: Remove the semicolon after the right parenthesis. [Note: As a result, the output statement will execute regardless of whether the condition in the `if` is true.]

b) Error: The relational operator `==>` is incorrect. Correction: Change `==>` to `>=`.

Q2. What does the following program print?

```
1  // Exercise 4.16: Mystery.java
2  public class Mystery
3  {
4      public static void main(String[] args)
5      {
6          int x = 1;
7          int total = 0;
8
9          while (x <= 10)
10         {
11             int y = x * x;
12             System.out.println(y);
13             total += y;
14             ++x;
15         }
16
17         System.out.printf("Total is %d\n", total);
18     }
19 } // end class Mystery
```

1

4

9

16

25

36

49

64

81

100

Total is 385

Q3. What does the following program print?

```
1 // Exercise 4.25: Mystery2.java
2 public class Mystery2
3 {
4     public static void main(String[] args)
5     {
6         int count = 1;
7
8         while (count <= 10)
9         {
10            System.out.println(count % 2 == 1 ? "****" : "+++++++");
11            ++count;
12        }
13    }
14 } // end class Mystery2
```

+++++++

++++

Be continue till 10

Q4. What does the following program print?

```
1 // Exercise 4.26: Mystery3.java
2 public class Mystery3
3 {
4     public static void main(String[] args)
5     {
6         int row = 10;
7
8         while (row >= 1)
9         {
10            int column = 1;
11
12            while (column <= 10)
13            {
14                System.out.print(row % 2 == 1 ? "<" : ">");
15                ++column;
16            }
17
18            --row;
19            System.out.println();
20        }
21    }
22 } // end class Mystery3
```


Output:

```
>>>>
<<<<<<<<<<<
>>>>>>
<<<<<<<<<<
>>>>
<<<<<<<<<<
>>>>
<<<<<<<<<<
>>>>
<<<<<<<<<<
>>>>
<<<<<<<<<<
```

Q5. What is wrong with the following statement? Provide the correct statement to add one to the sum of x and y.

```
System.out.println(++(x + y));
```

Answer:

```
System.out.println(x + y + 1);
```

Q6. Find the error in each of the following code segments, and explain how to correct it:

```

a) i = 1;
   while (i <= 10);
       ++i;
   }
b) for (k = 0.1; k != 1.0; k += 0.1)
    System.out.println(k);
c) switch (n)
    {
        case 1:
            System.out.println("The number is 1");
        case 2:
            System.out.println("The number is 2");
            break;
        default:
            System.out.println("The number is not 1 or 2");
            break;
    }
d) The following code should print the values 1 to 10:
    n = 1;
    while (n < 10)
        System.out.println(n++);

```

Int i=1;

While(i<=10){

++I;

}

b)

Q8. Find and correct the error(s) in each of the following segments of code:+

```

a) For (i = 100, i >= 1, i++)
    System.out.println(i);
b) The following code should print whether integer value is odd or even:
    switch (value % 2)
    {
        case 0:
            System.out.println("Even integer");
        case 1:
            System.out.println("Odd integer");
    }
c) The following code should output the odd integers from 19 to 1:
    for (i = 19; i >= 1; i += 2)
        System.out.println(i);

```

d) The following code should output the even integers from 2 to 100:

```
counter = 2;
do
{
    System.out.println(counter);
    counter += 2;
} While (counter < 100);
```

a)for (i = 100; i >= 1; i--){
}

b)

Q9. What does the following program do?

```
1 // Exercise 5.10: Printing.java
2 public class Printing
3 {
4     public static void main(String[] args)
5     {
6         for (int i = 1; i <= 10; i++)
7         {
8             for (int j = 1; j <= 5; j++)
9                 System.out.print('@');
10
11             System.out.println();
12         }
13     }
14 } // end class Printing
```

@@@@@

@@@@@

@@@@@

@@@@@

@@@@@

@@@@@

@@@@@

@@@@@

@@@@@

@@@@@